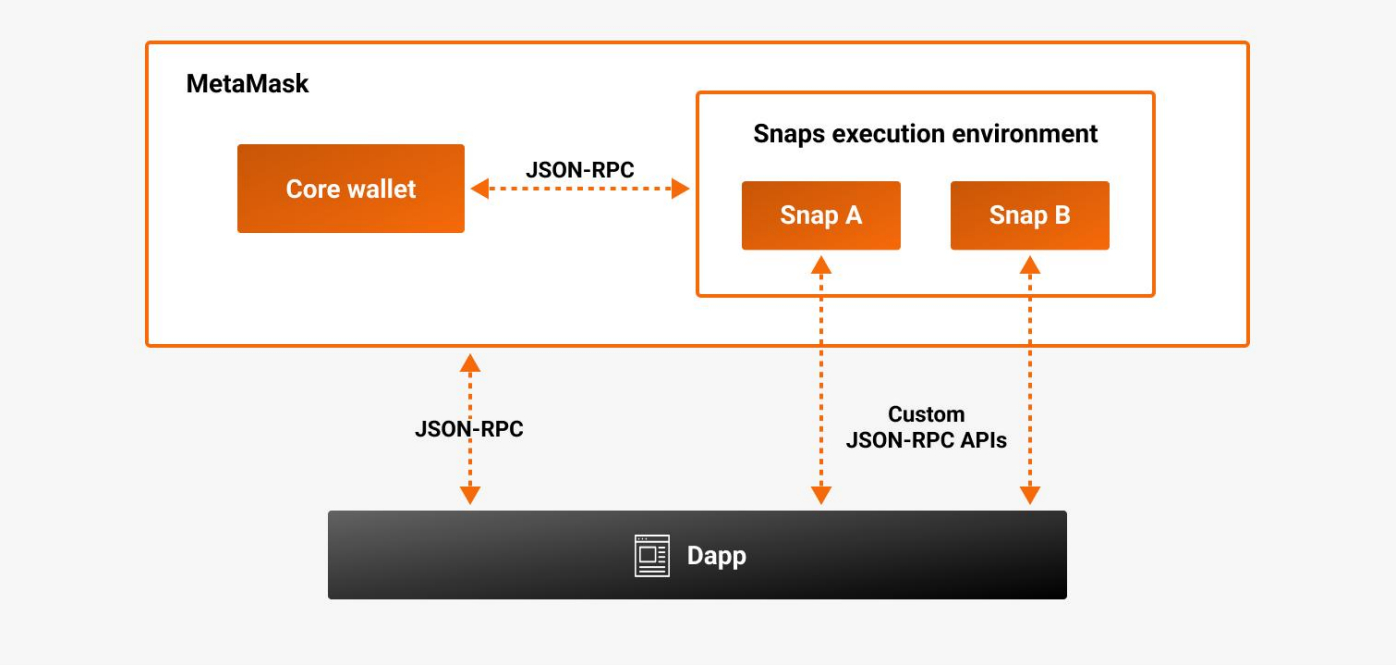
General Description

This document describes how MetaMask Gardio Snap Works and its Architecture, Sequence Diagram, and Modifications

Contents

1 Architecture

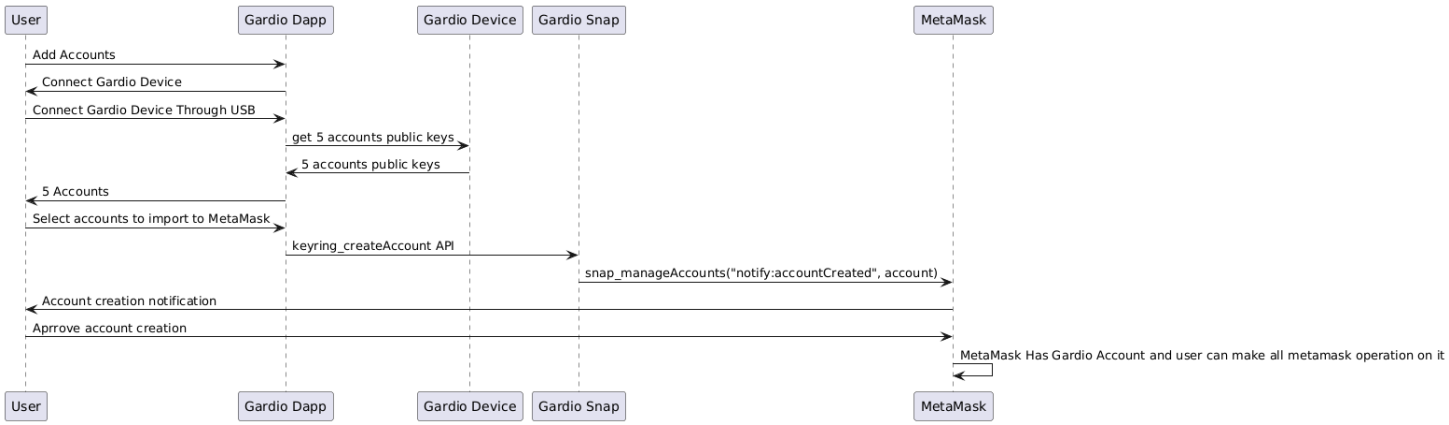
- MetaMask Snaps is an open source system that allows anyone to safely create a mini app that runs inside the MetaMask extension, enabling new web3 end user experiences. For example, a Snap can add support for different blockchain networks, add custom account types, or provide additional functionality using its own APIs. This allows MetaMask to be used with a far more diverse set of protocols, dapps, and services.
- A Snap can communicate with MetaMask using the Snaps API
- Dapps can use the Wallet API for Snaps to install and communicate with Snaps.
- Gardio snap uses the Keyring API in communication between the Dapp, snap, and Metamask
    - This section provides a detailed reference on the Keyring API methods, objects, and events that enable custom EVM accounts. The Keyring API consists of:
        - Account Management API - An API for dapps to communicate with account management Snaps. Dapps can manage accounts and signature requests using this API.
        - Chain Methods API - An API that contains chain-specific EVM methods that account management Snaps can choose to implement to support dapp requests from custom accounts.
- The following diagram outlines the high-level architecture of the Snaps system:



2 Sequence Diagram

2.1 Account creation flow

- Once the Snap is installed in MetaMask, the user can use the Dapp to fetch the accounts from the gardio device and import it to MetaMask
- The flow looks like the following:



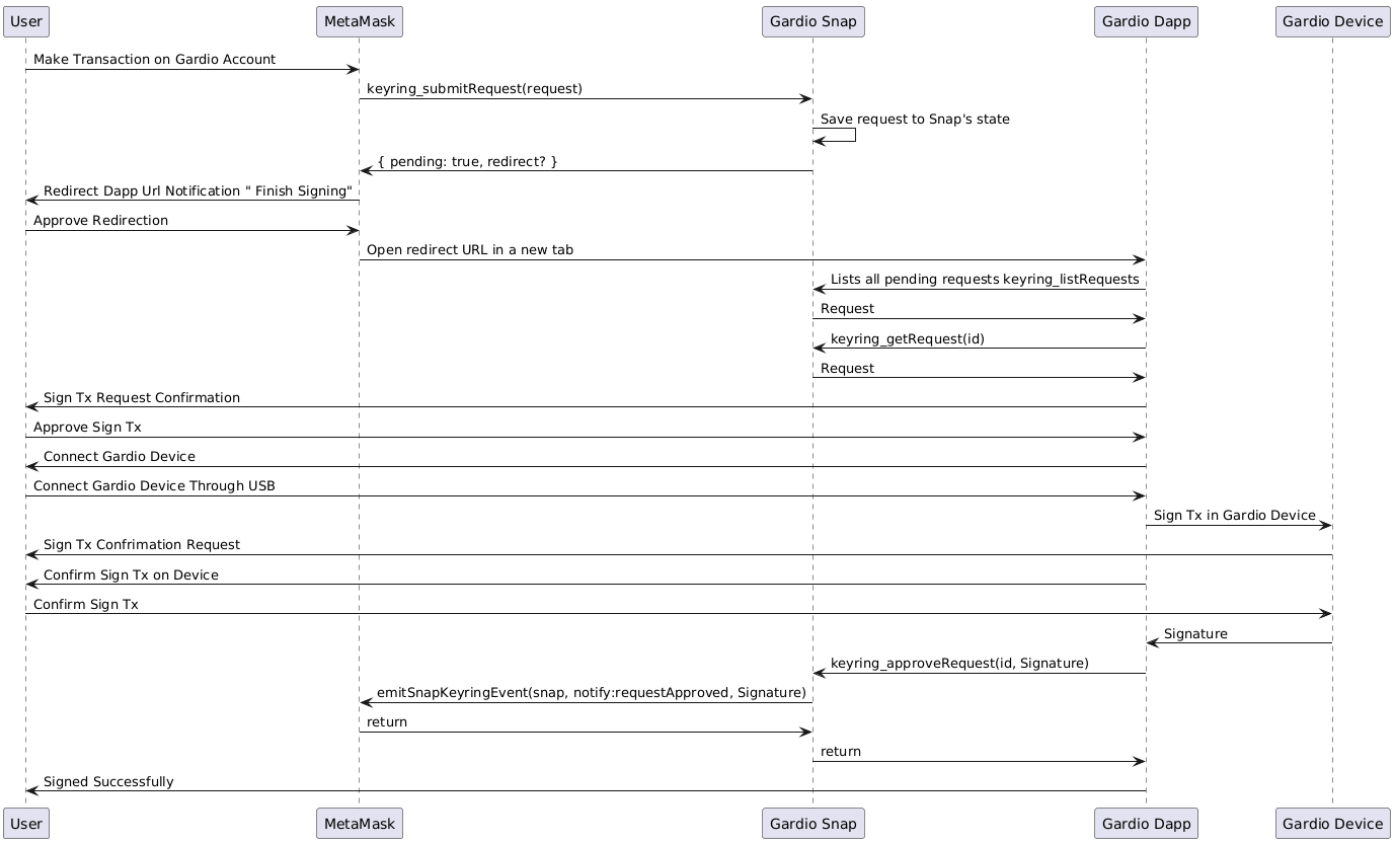
- The dapp creates an account using **keyring_createAccount**.
- The Snap keeps track of the accounts that it creates using **snap_manageState**.
- Once the Snap has created an account, it notifies MetaMask using AccountCreated Keyring Event
- Once the Snap has created an account, that account can be used to sign messages and transactions.
- MetaMask snap official flow

2.2 Account Update flow

- Once the Snap imports the account in MetaMask, the user can use the Dapp to update the account methods
- The flow looks like the following:
    - The dapp call keyring_updateAccount keyring API in Snap
    - The snap call KeyringEvent.AccountUpdated Keyring Event to notify Metamask by updating
    - Metamask update the account

2.3 Transaction flow

- In MetaMask, the Keyring API supports two flows for handling requests: synchronous and asynchronous.
- We use the asynchronous flow in gardio Snap to sign the transaction in the gardio device wallet
- Once the Snap Gardio Account is imported into MetaMask, the user can make any transaction in the Gardio Accounts from MetaMask
- The flow looks like the following:



- The user initiates a Transaction in MetaMask Extension in Gardio Accounts
- MetaMask calls keyring_submitRequest.
- Since the Snap doesn't answer the request directly, it stores the pending request in its internal state using snap_manageState
- The Gardio Snap sends a { pending: true, redirect? } response to indicate that the request will be handled asynchronously.
- This response contains a redirect Dapp URL that MetaMask will open in a new tab to allow the user to interact with the Gardio Dapp.
- The Gardio dapp gets the Snap's pending request using keyring_getRequest.
- The Gardio dapp sign the tx in Device and resolves the request using keyring_approveRequest,
- the Snap resolves the request and notifies MetaMask using RequestApproved Keyring Event with Signature

- After approval, MetaMask calls.
- Since the Snap doesn't answer the request directly, it stores the pending request in its internal state using. The Snap sends a { pending: true, redirect? } response to indicate that the request will be handled asynchronously. This response can optionally contain a redirect URL that MetaMask will open in a new tab to allow the user to interact with the Snap companion dapp.
- This flow is for signing a message, too, but starting the sign message from the Metamask test Dapp
    - https://metamask.github.io/test-dapp/
    - Use User Guide document as reference to use it

3 Modifications

3.1 Account Creation Modification

- Our snap official metamask reference is this example repo
  - https://github.com/MetaMask/snap-simple-keyring
- **Snap API Modification**
  - **keyring_createAccount**
    - In MetaMask example
      - Dapp call it with paramter privateKey as optional paramter
      - Snap Get the keypair (privateKey, address) if the privateKey passed as paramter
      - Snap Get the address only if there no privateKey in paramters
      - Import the account in metamask
    - In Gardio Snap
      - Dapp call it with parameters (account address, account name
      - Import the account in metamask

3.2 Transaction Modification

- MetaMask flow
  - Asynchronous transaction flow
  - The user create the transaction request from the Dapp
  - The Dapp send the request to metamask
  - Metamask review approve on user
  - The user approve in metamask
  - The metamask send the transaction request to the snap
- Gardio snap flow
  - Gardio Transaction flow
  - The user create the transaction request from metamask
  - The metamask send the transaction request to the snap

4 References

4.1 MetaMask

4.1.1 Snap Document

- **https://docs.metamask.io/snaps/**

4.1.2 Snap Simple MetaMask Example Repo

- https://github.com/MetaMask/snap-simple-keyring

4.2 Jira Tickets

- https://thirdwayv.atlassian.net/issues/?jql=project%20%3D%20%27CRYP%27%20AND%20textfields%20~%20%22MetaMask%20Snap%22

4.3 Wasm

- https://github.com/emscripten-core/emsdk
- https://thirdwayv.atlassian.net/wiki/spaces/CRYP/pages/2840461313/Automation+test+bringup

4.4 Solana Support

- https://github.com/MetaMask/snap-solana-wallet
- https://www.npmjs.com/package/@metamask/solana-wallet-snap